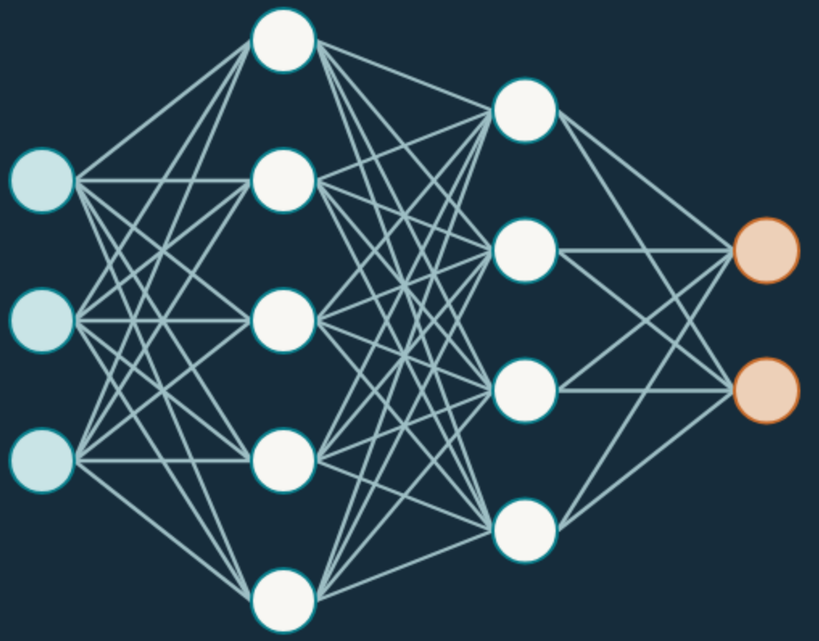
Multilayer perceptron neural networks

A tutorial in mathematics, data, and working code



Weights and nonlinear functions
Learning from labeled examples
Training, testing, and prediction

20 slides with detailed notes
Python + NumPy laboratory included

Supervised learning: examples and labels

The training set pairs each input x with a target y that tells us what to predict.

Temperature	Pressure	Catalyst?	Material	Label
520 K	3 bar	true	A	phase_A
430 K	7 bar	false	B	phase_B
610 K	5 bar	true	C	phase_C

Features x

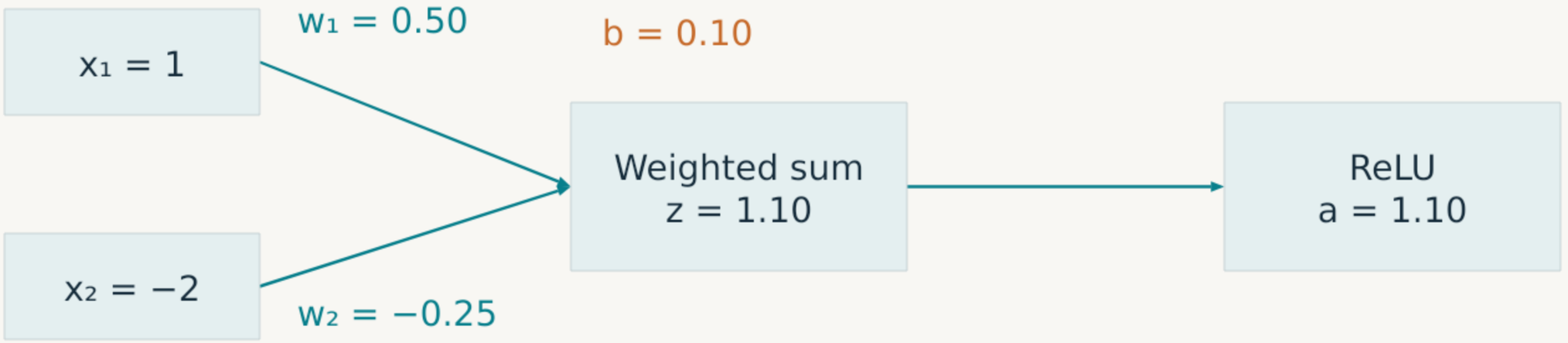
Target y

Classification predicts a category.
Example: one of three material phases.

Regression predicts a number.
Example: a continuous response in measured units.

One neuron: weights, bias, and activation

A weight scales an input. The bias shifts the response even when every input is zero.



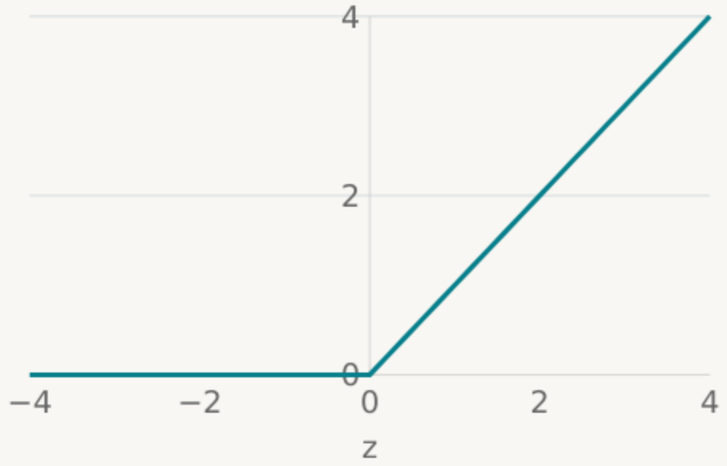
$$z = w_1x_1 + w_2x_2 + b$$

$$z = 0.50(1) + (-0.25)(-2) + 0.10 = 1.10$$

Activation functions introduce nonlinearity

Without a nonlinear activation, any stack of dense layers collapses to one affine map.

ReLU



max(0, z)

Simple hidden-layer default.
Zero derivative for z < 0.

tanh



tanh(z)

Outputs between −1 and 1.
Large |z| gives small gradients.

Sigmoid

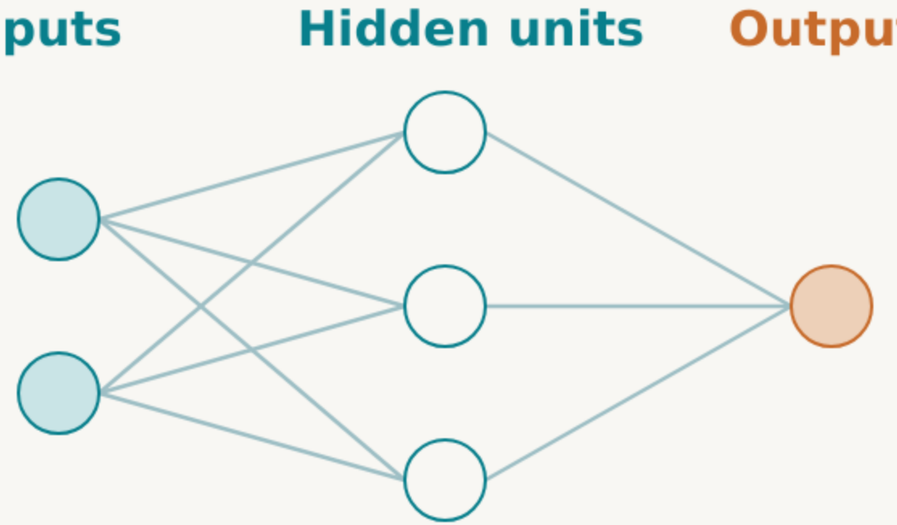


1 / (1 + exp(−z))

Outputs between 0 and 1.
Useful for binary probabilities.

Layers learn combinations of features

XOR returns 1 when exactly one of two Boolean inputs is true.



x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Each connection has a weight.
Each hidden or output unit has a bias.

A single straight boundary cannot separate these classes. Hidden nonlinear units can.

The forward pass as matrix mathematics

Rows are examples. Columns are features or units. The same weights serve every row.

$$A^{(0)} = X \qquad Z^{(l)} = A^{(l-1)}W^{(l)} + b^{(l)} \qquad A^{(l)} = \varphi(Z^{(l)})$$

Quantity	Shape	Example: batch of 64
Input X	B × d	64 × 11
First weights W^1	d × h_1	11 × 32
First hidden activations	B × h_1	64 × 32
Second hidden activations	B × h_2	64 × 16
Output logits	B × K	64 × 3

11 → 32 → 16 → 3

(11×32 + 32)
+ (32×16 + 16)
+ (16×3 + 3)
= 963 parameters

The bias broadcasts across all B rows. The final layer produces logits or continuous values.

Raw input types become numerical features

An explicit schema preserves meaning when the network receives only numbers.

Input type	Representation in this lab	Reason
Numerical	Training median; then $(x - \mu) / s$	Comparable scales improve optimization.
Boolean / logical	false = 0, true = 1	Preserves two distinct logical values.
Categorical	One-hot vector from training vocabulary	Avoids inventing an order among names.
Missing value	Imputation plus a missingness flag	Unknown is different from a measured zero.
Unseen category	Dedicated unknown-category position	The input width stays fixed at inference.

Example: material B becomes [0, 0, 0, 1, 0]

Column order: missing, unknown, A, B, C. Fit μ, s, medians, and vocabulary on training rows only.

Labels, splits, and leakage

Training changes weights. Validation chooses settings. Testing estimates performance on held-out data.

TRAIN about 70%	VALIDATION about 15%	TEST about 15%
Fit preprocessing. Compute gradients. Update weights.	Choose learning rate. Select the best epoch. Set any decision threshold.	Score the frozen model. Compare with a baseline. Do not tune on these scores.

Labels must match the task

Regression: measured numbers and units.
Classification: stable names mapped to class IDs.

Group related samples together.
For forecasting, split by time.

Regression predicts continuous values

A linear output allows any real value. The loss measures distance from the observed target.

$$\hat{y} = f_\theta(x) \qquad L = (1/B)\,\Sigma_i\,(\hat{y}_i - y_i)^2$$

Observed y	Predicted ŷ	Residual ŷ − y	Squared error
10	12	+2	4
20	17	−3	9
30	31	+1	1

MSE = 14/3 = 4.67

RMSE = √MSE = 2.16

MAE = 6/3 = 2.00

RMSE and MAE use the target’s original units.
MSE penalizes a large error more strongly than a small one.

Classification: logits, probabilities, and labels

A logit is an unconstrained score. The output interpretation and loss must agree.

Task	Output / target	Probability and loss
Binary	1 logit; target 0 or 1	Sigmoid + binary cross entropy
Multiclass	K logits; one class ID per row	Softmax + cross entropy
Multilabel	K logits; K independent 0/1 labels	K sigmoids + binary cross entropy

$$p_k = \exp(z_k) / \Sigma_j \exp(z_j) \qquad L = -\log p_Y$$

Example: $z = [2, 1, 0]$ $p \approx [0.665, 0.245, 0.090]$

For true class $y = 0$, $L \approx 0.408$. The training loss accepts logits directly.

Backpropagation and the chain rule

The chain rule carries the loss sensitivity backward through the layers.

$$\partial L/\partial W^{l} = (A^{l-1})^{T}\,\delta^{l} \qquad \partial L/\partial b^{l} = \text{sum_rows}(\delta^{l})$$

$$\delta^{l-1} = [\delta^{l}\,(W^{l})^{T}] \odot \varphi'(Z^{l-1})$$

δ is the derivative with respect to a layer’s preactivation. ⊙ means elementwise multiplication.

One numerical update

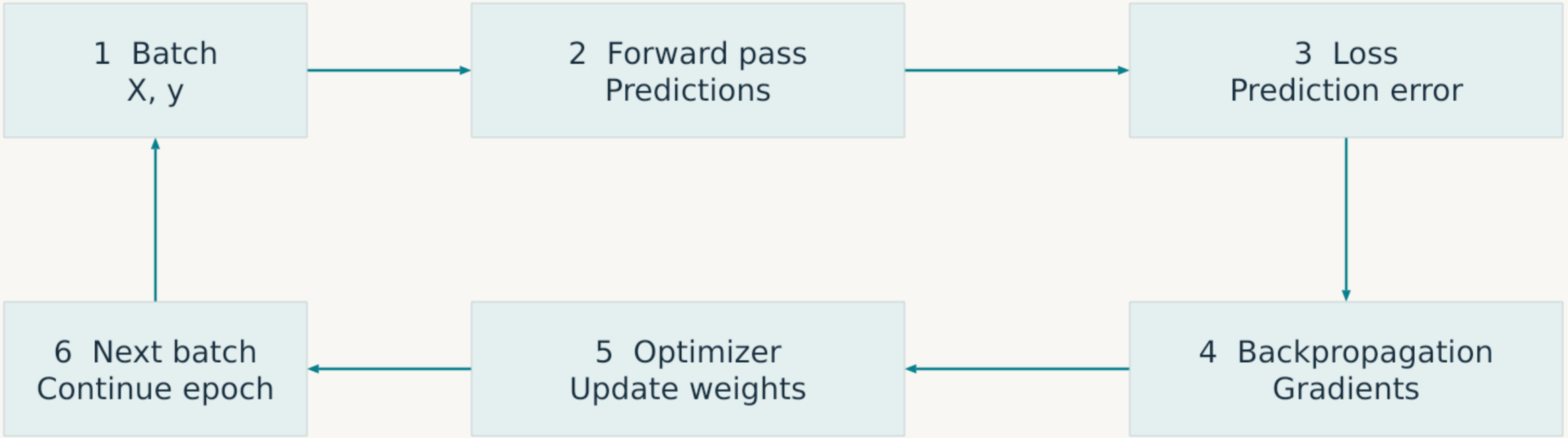
Reuse slide 3: x = [1, −2], $\hat{y}$ = 1.10.
Let y = 1.05 and L = ½($\hat{y}$ − y)².
Then ∂L/∂w = [0.05, −0.10], ∂L/∂b = 0.05.

Learning rate η = 0.1

w = [0.495, −0.240]
b = 0.095
New prediction = 1.07

The training loop

Each minibatch produces a loss, gradients, and one optimizer update.



At the end of each epoch: evaluate validation loss, save the best weights, and check the stopping rule.

Optimizers turn gradients into parameter updates

The gradient supplies a direction. The optimizer determines how to use it.

SGD with momentum

$$v_t = \mu v_{t-1} + g_t$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

Momentum averages a history of directions.
A learning rate η scales every update.
Plain SGD sets $\mu = 0$.

AdamW

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t$$

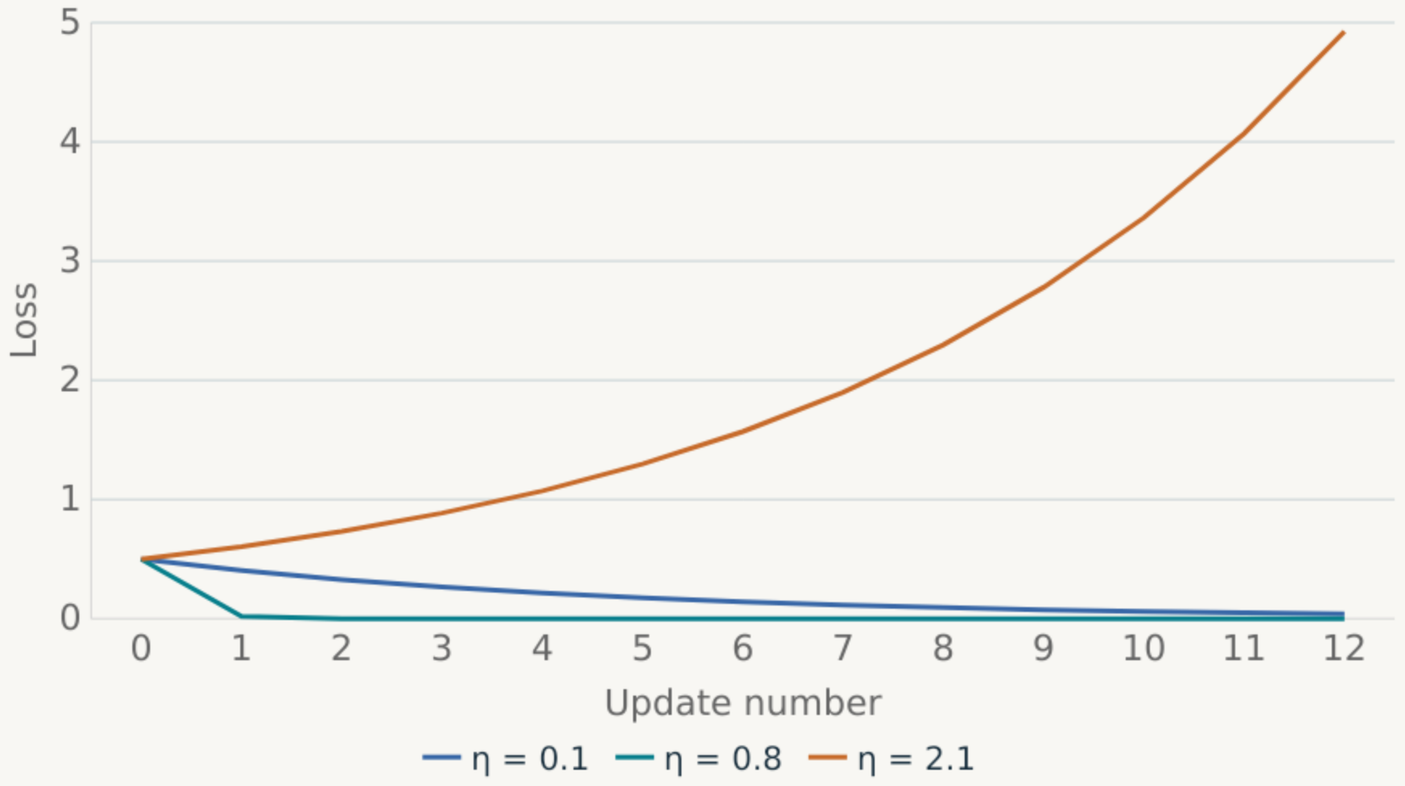
$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2$$

$$\theta_{t+1} = (1-\eta\lambda)\theta_t - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t}+\varepsilon)$$

$\hat{m}$ and $\hat{v}$ correct the initial bias.
λ controls decoupled weight decay.
The lab excludes biases from decay.

Learning rate controls the size of each step

An exactly solvable toy loss shows slow progress, rapid progress, and divergence.



$$L(w) = \tfrac{1}{2}w^2$$

$$w_{t+1} = (1-\eta)w_t$$

Starting value: $w_0 = 1$.
These curves follow the equation;
they are not MLP measurements.

Compare MLP learning rates on the same validation split.
Smaller steps can help late in training.

Batches, steps, and epochs

An epoch visits the training set once. A step performs one parameter update.

Quantity	Definition	Example
Training examples N	Number of rows available for fitting	1,680
Batch size B	Examples used for one gradient	64
Steps per epoch	ceil(N / B), retaining the last batch	27
Final batch	Rows left after 26 full batches	16
100 epochs	100 × 27 optimizer steps	2,700

Small batches use less activation memory and give noisier gradient estimates.

Large batches use matrix hardware efficiently.
Changing batch size may require retuning η.

The model in executable NumPy

The implementation exposes the same operations as the mathematics.

```
net = MLP([11, 32, 16, 3],
          activation="relu", seed=7)

# Inside the forward pass
z = a @ W + b
a = np.maximum(z, 0)  # hidden layer

# Inside backpropagation
dW = a_prev.T @ dz
db = dz.sum(axis=0, keepdims=True)
da_prev = dz @ W.T
```

Shapes tell the story

X: [B, 11]
Hidden: [B, 32]
Hidden: [B, 16]
Logits: [B, 3]

mlp.py contains the network, loss derivatives, and optimizers. data.py handles input meaning.

A complete training and validation harness

Validation selects the checkpoint. The saved model includes its preprocessing and label mapping.

```
for epoch in range(epochs):
    order = rng.permutation(len(X))
    for start in range(0, len(X), B):
        ids = order[start:start+B]
        loss, grads = net.loss_grad(
            X[ids], y[ids], task)
        opt.step(net.params, grads)

    val_loss = net.loss(X_val, y_val, task)
    # Save a copy if validation improves.
    # Stop after patience stale epochs.
```

weights.npz
Learned numerical parameters

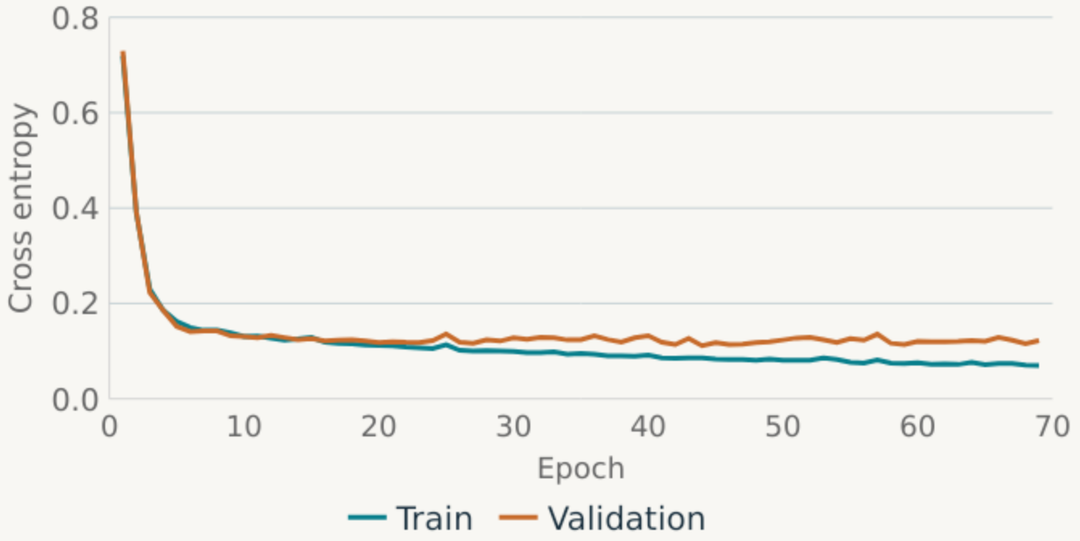
metadata.json
Schema, transforms, labels, split

history.csv
Losses at each epoch

Held-out testing: measured results

These results come from the included synthetic data and CPU runs, with model seed 7.

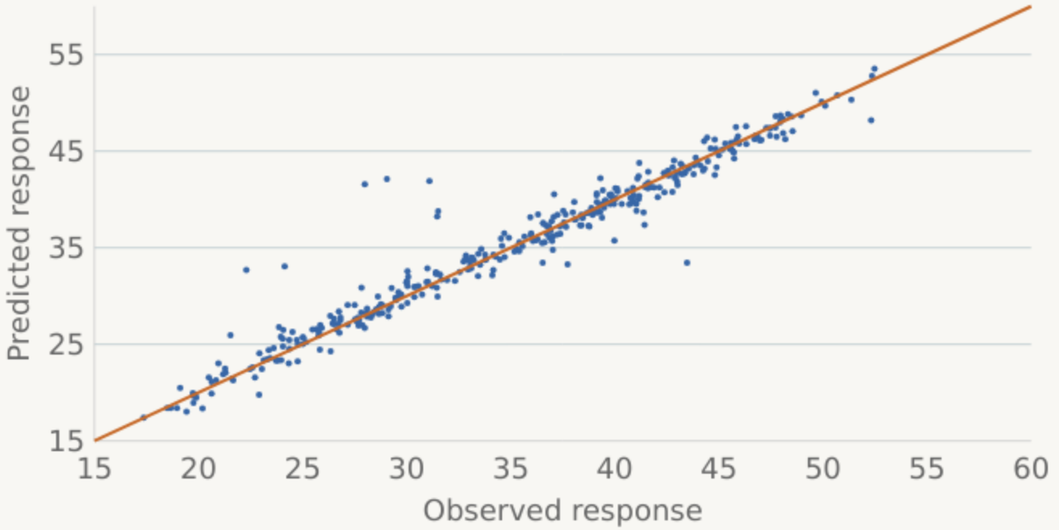
Classification



94.5% accuracy macro-F1 0.930

Majority baseline: 48.2% accuracy
Best validation epoch: 44; test rows: 363

Regression



RMSE 1.90 R^2 0.947

Mean baseline: RMSE 8.33
Best validation epoch: 76; test rows: 360

Diagnostics, regularization, and tests

A decreasing training loss is useful evidence, but generalization needs independent checks.

Observation	Possible explanation	Next check
Training and validation both poor	Underfitting, bad scaling, or optimization	Fit a tiny batch; inspect data and gradients.
Train improves; validation worsens	Overfitting to the training examples	Earlier stopping; smaller model; more data.
Loss jumps or becomes nonfinite	Learning rate, extreme values, or unstable math	Reduce η; inspect scales and stable losses.
High accuracy; a rare class fails	Class imbalance hides errors	Confusion matrix, recall, macro-F1.

Regularization

Weight decay and early stopping are implemented.
Dropout is a useful extension, omitted here.

The included tests

Gradient checks and data checks.
Reload and learning tests.

Laboratory and further reading

Run the full experiment, then change one assumption at a time.

```
python -m pip install -r requirements.txt
python run_all.py

# Separate held-out evaluation
python evaluate.py --csv data/classification.csv \
    --run runs/classification
```

Exercises

1. Compare hidden widths 8, 32, and 64.
2. Compare learning rates on validation data.
3. Add your CSV and an explicit input schema.
4. Inspect per-class errors and regression residuals.

References

Goodfellow et al., Deep Learning, Ch. 6
Stanford CS231n neural-network notes
Kingma & Ba, Adam (2015)
Loshchilov & Hutter, AdamW (2019)
scikit-learn: data leakage guidance